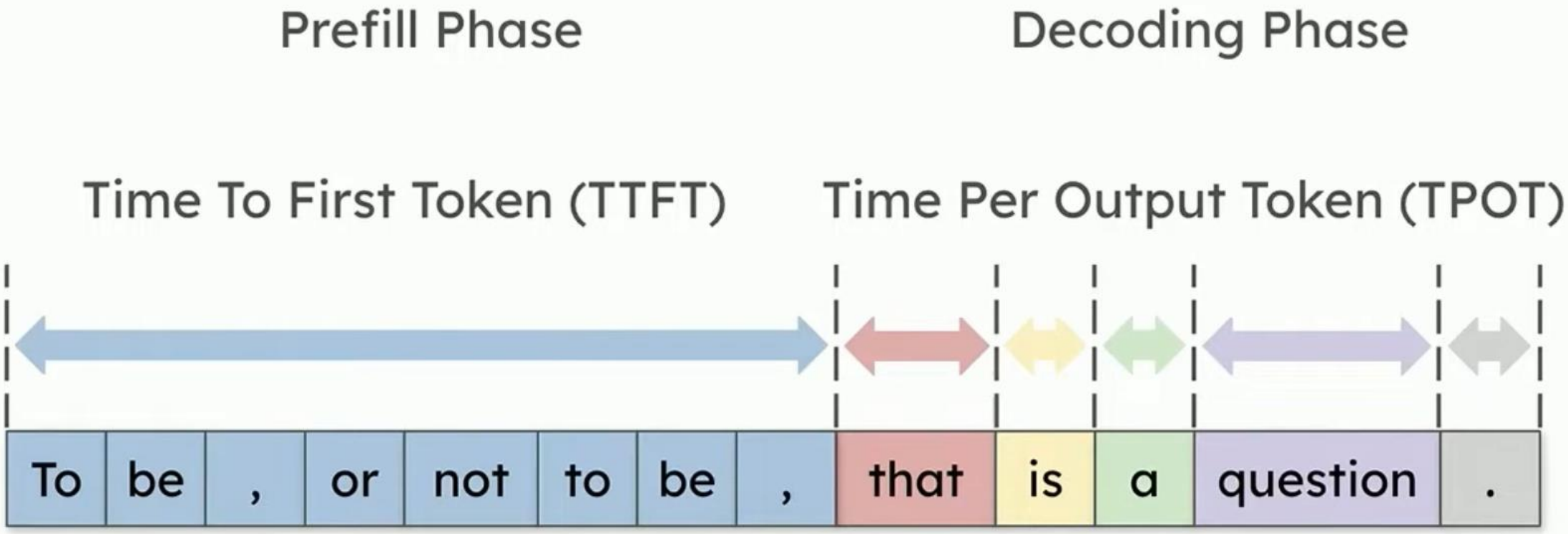
DistServe

Danyang Zhuo

Duke University

Some materials are from DistServe's OSDI Slides

LLM Inference Latency



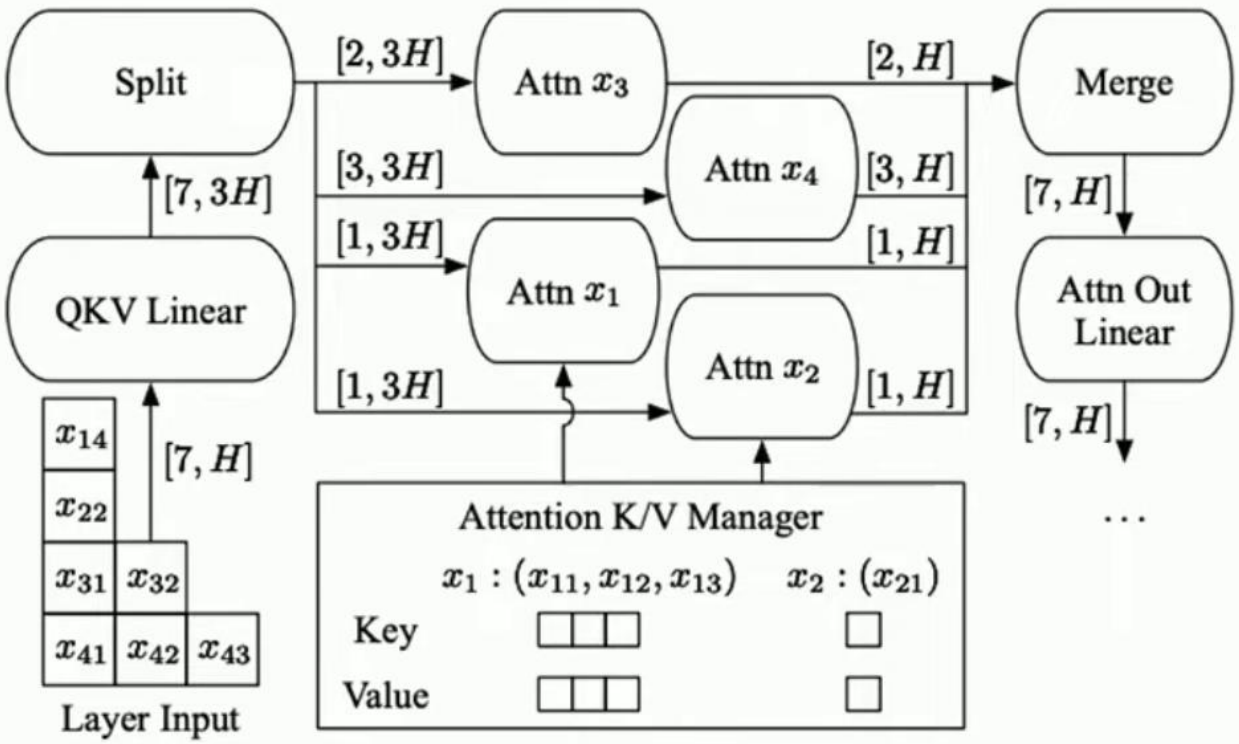
Latency = TTFT + TPOT * #Token

Different apps have various latency requirements

	TTFT	TPOT
Chat	Low (< 1s)	Match read speed (~ 100ms)
Search	Very Low (~ 200ms)	Match read speed (~ 100ms)
Program	Very Low (~ 200ms)	Very Low (~ 50ms)

Existing systems maximize the overall throughput

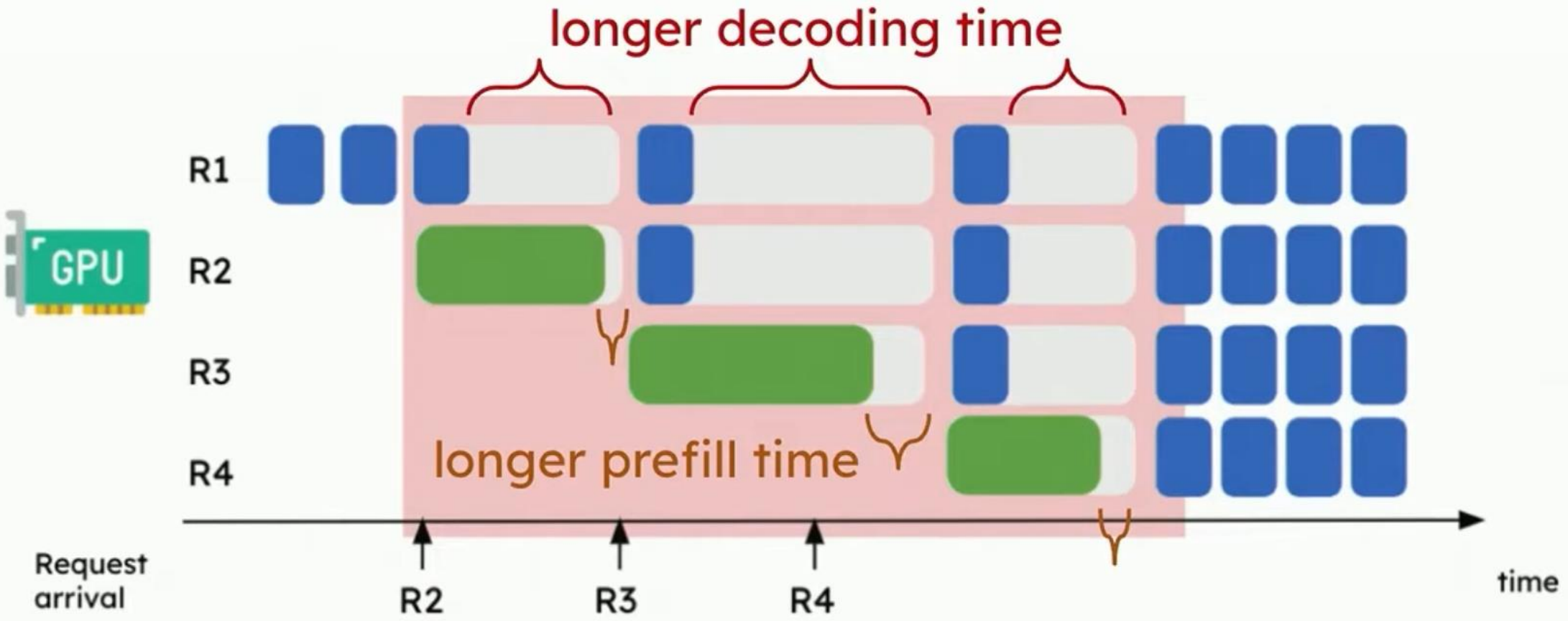
Existing approach: Batch requests in prefill and decoding phase together

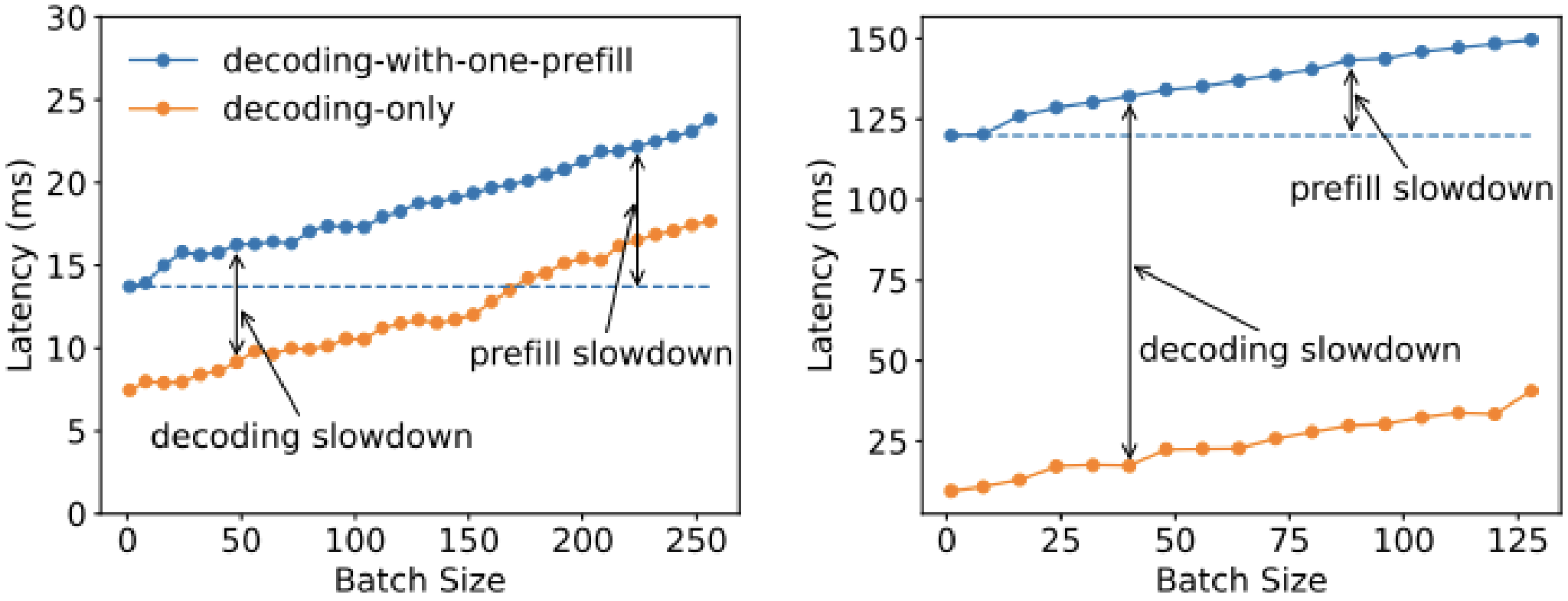


Source: Gyeong-In Yu, et al. "Orca: A Distributed Serving System for Transformer-Based Generative Models"

Problem 1: Prefill-Decoding Interference

Batching prefill and decoding phase together hurt both TTFT and TPOT





(a) Input length = 128 (b) Input length = 1024

Figure 2: Batch execution time when serving a 13B LLM as batch size increases. Compared between a decoding-only batch and the batch adding one more prefill job.

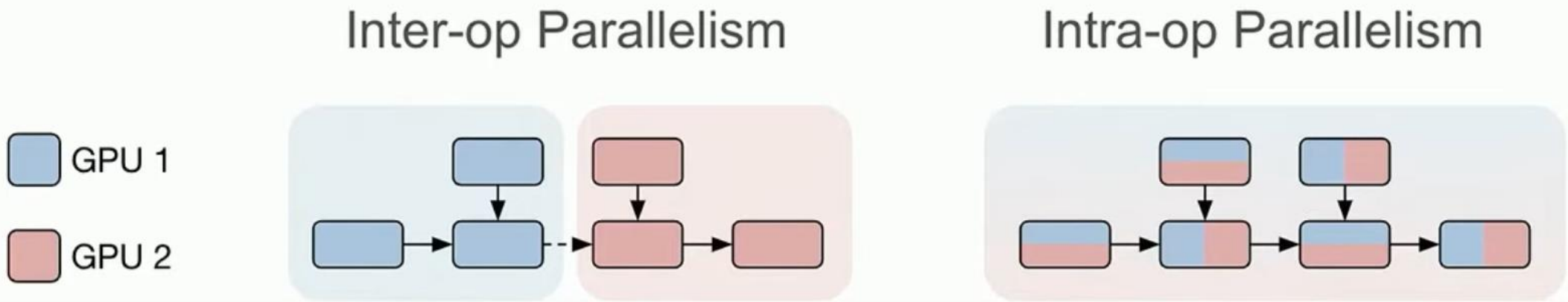
Chunked prefill

- This technique alleviates the slowdown of the decoding job caused by the long prefill job, but it does not eliminate it.

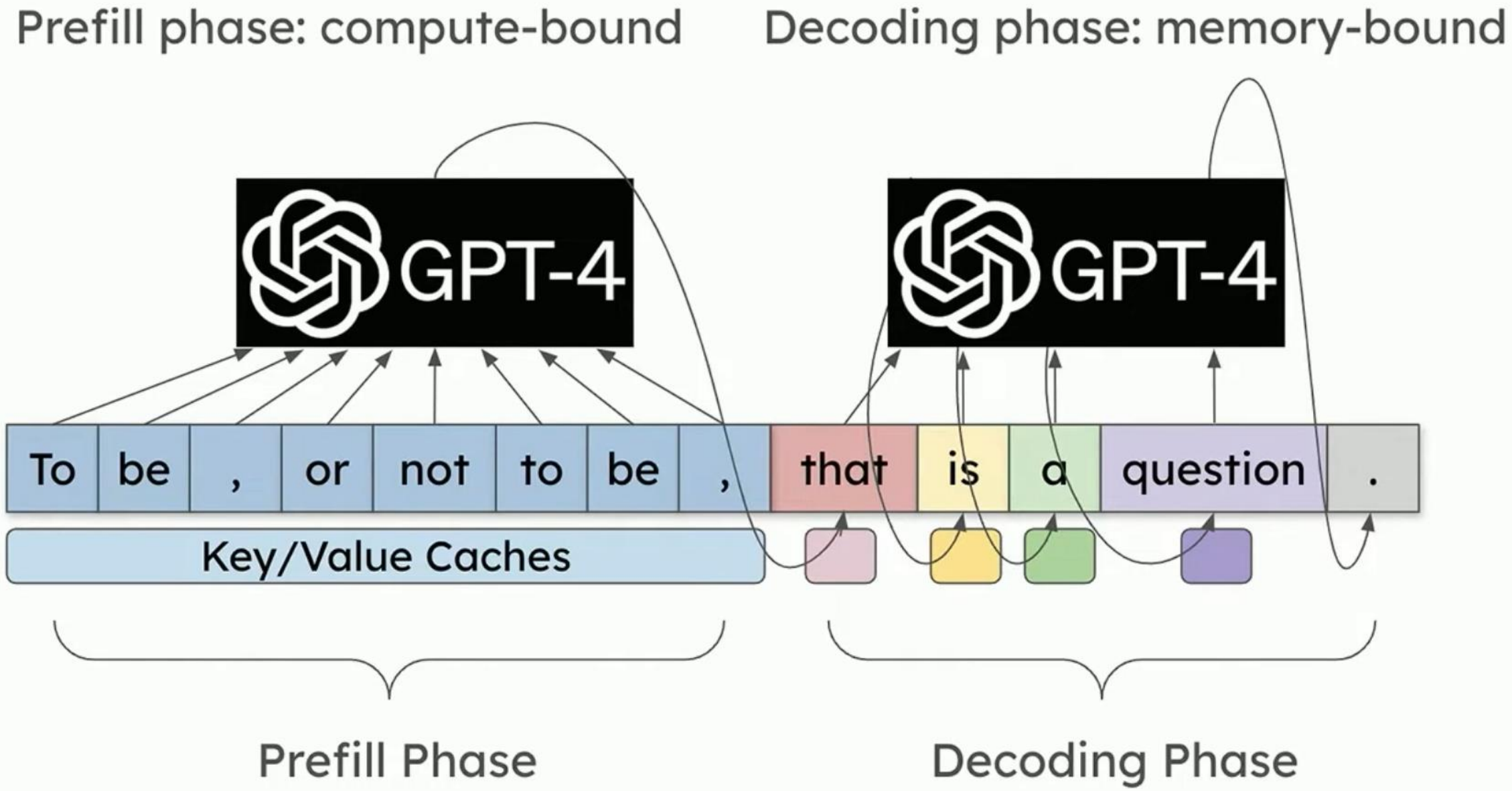
- It results in an extra overhead for the prefill job which cannot be easily mitigated by adjusting the chunk size.
  - Small chunk size -> Low GPU utilization
  - Large chunk size -> High TPOT

Problem 2: Resource and Parallelism Coupling

Batching the two phases make them share the same parallel strategy

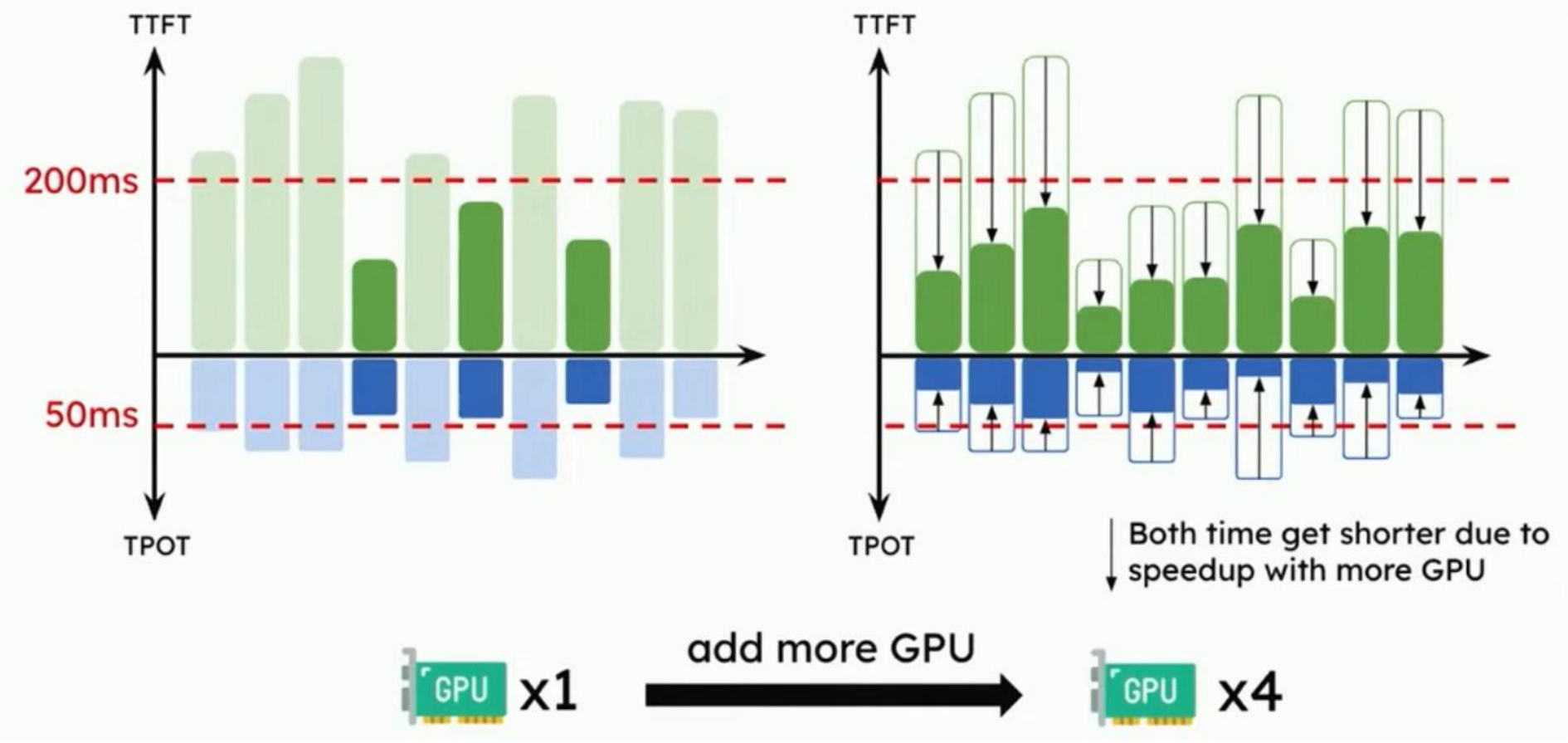


Problem 2: Resource and Parallelism Coupling



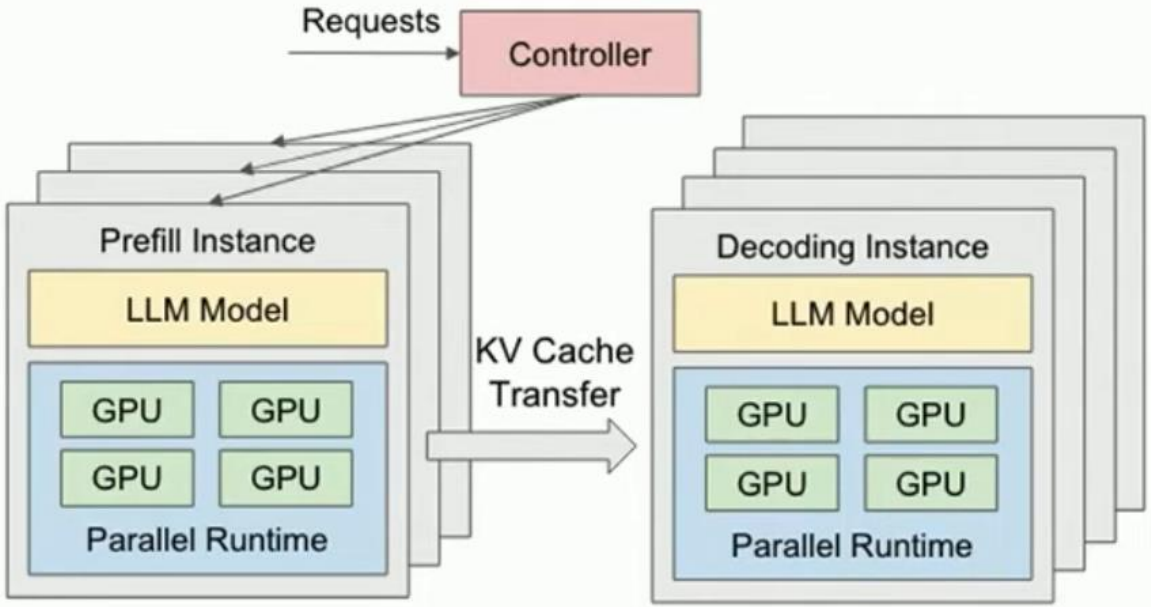
Problem 2: Resource and Parallelism Coupling

Coupling leads to overprovision resources to meet the more demanding SLO

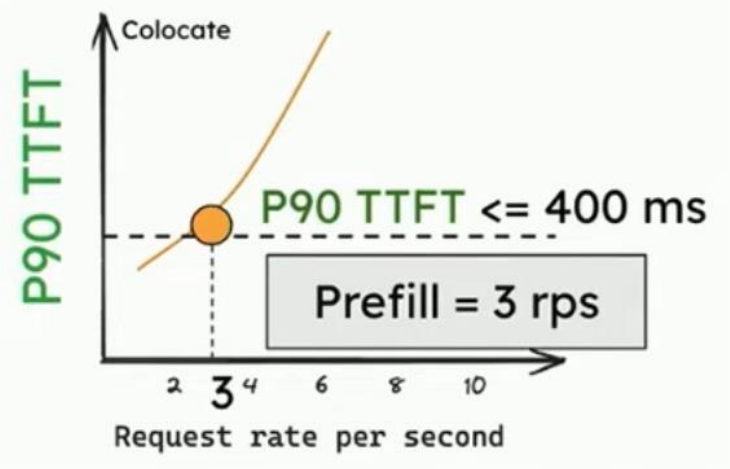


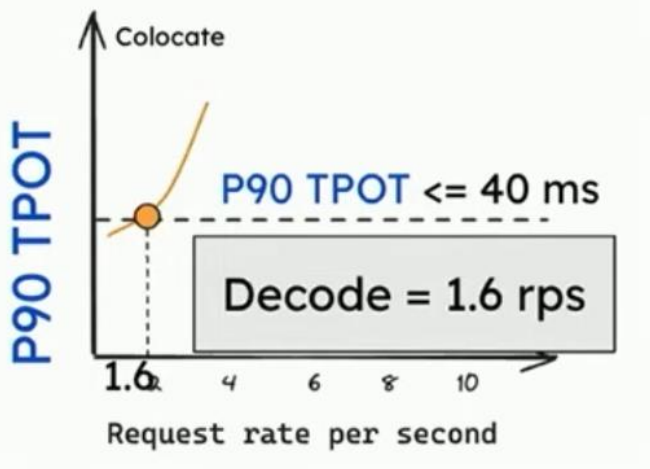
Opportunity: Disaggregating Prefill and Decoding

- Prefill-Decoding interference is immediately eliminated
- Naturally divide the SLO satisfaction problem into two optimizations:
  - Prefill instance optimizes for TTFT.
  - Decoding instance optimizes for TPOT.
  - Choose the most suitable parallelism and resource allocation for each phase.



Opportunity: Disaggregting Prefill and Decoding



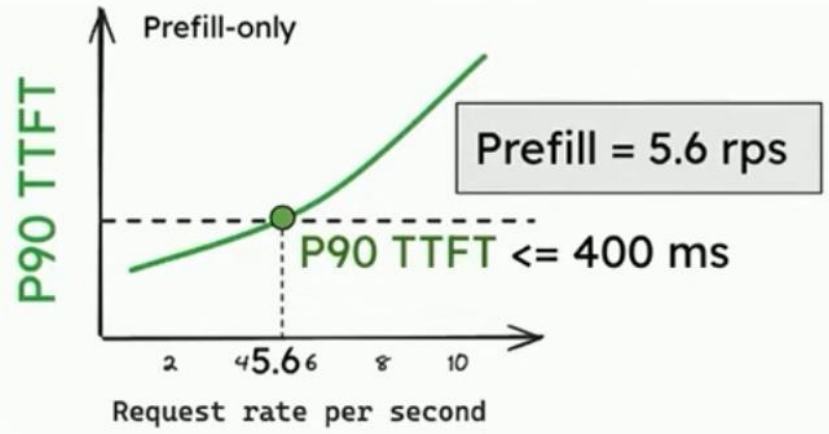


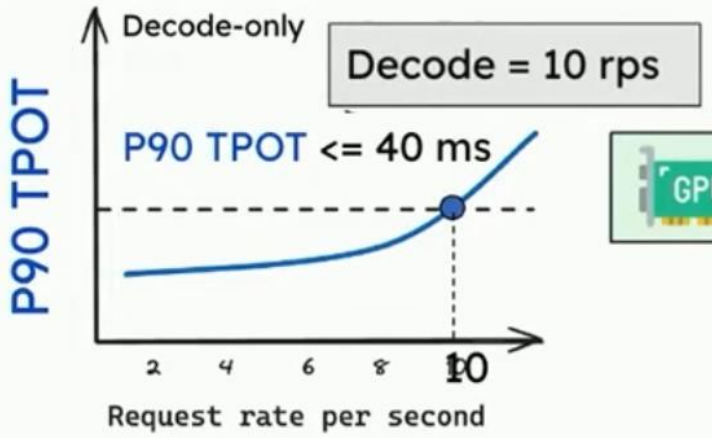
Colocation

Max System rps

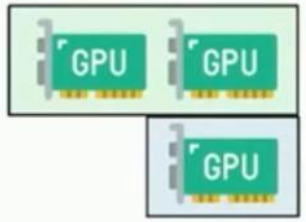
= Min(Prefill, Decode)

= 1.6 rps / GPU





Disaggregation (2P1D)



Max System rps

= Min (5.6 x **2**, 10) rps / 3 GPU

= 3.3 rps / GPU

Challenges of Disaggregation

- Communication overhead for KV-Cache transmission
- The optimization target — ***per-GPU goodput***, is difficult to optimize:
  - the workload pattern
  - SLO requirements
  - parallelism strategies
  - resource allocation
  - network bandwidth

DistServe Overview

Definition of Placement:

(1) parallelism strategy for prefill/decoding instance
(2) the number of each instance to deploy
(3) how to place them onto the physical cluster

Featured algorithms:

- Placement for High Node-Affinity Cluster
- Placement for Low Node-Affinity Cluster
- Online Scheduling Optimization

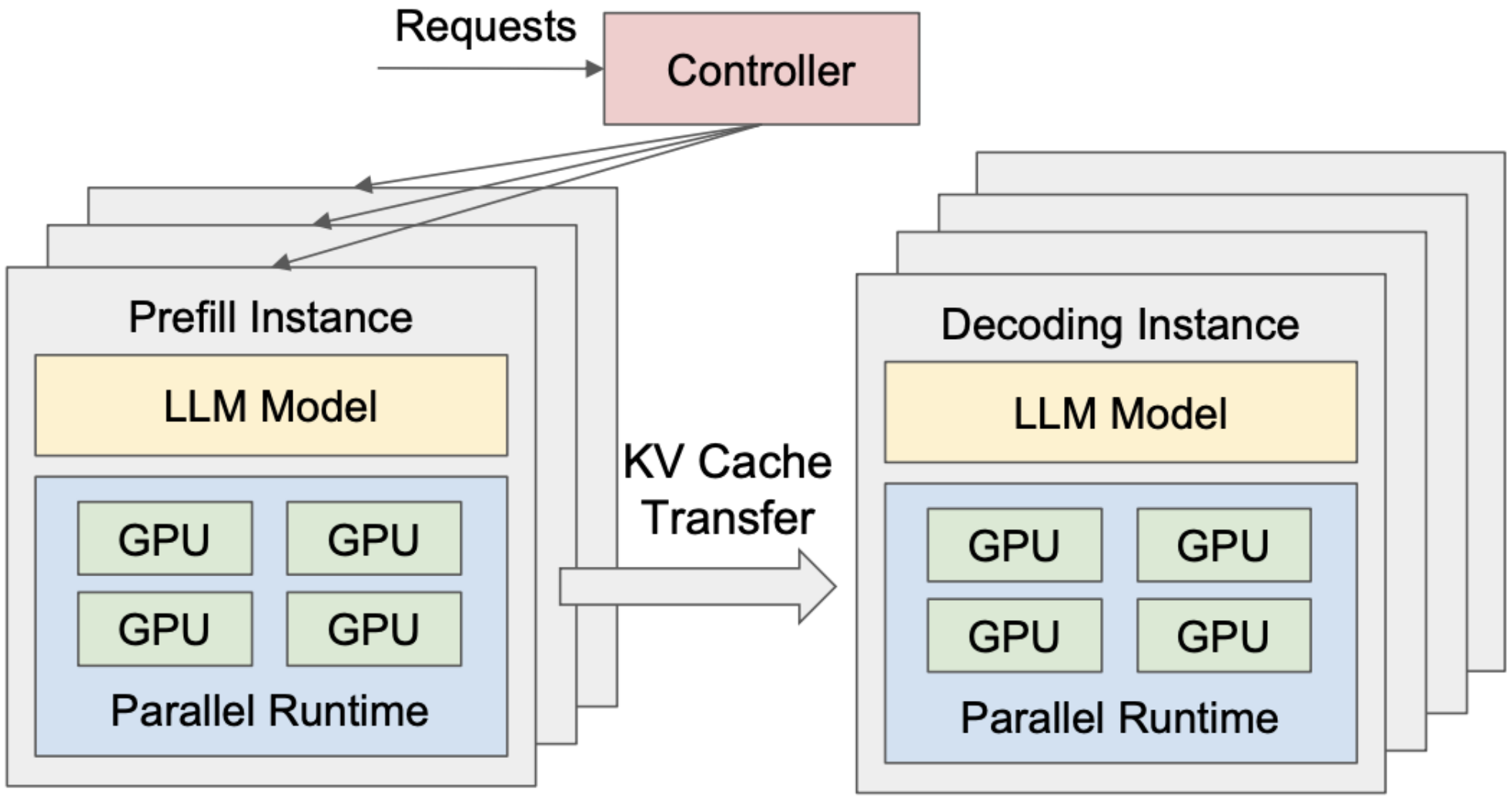
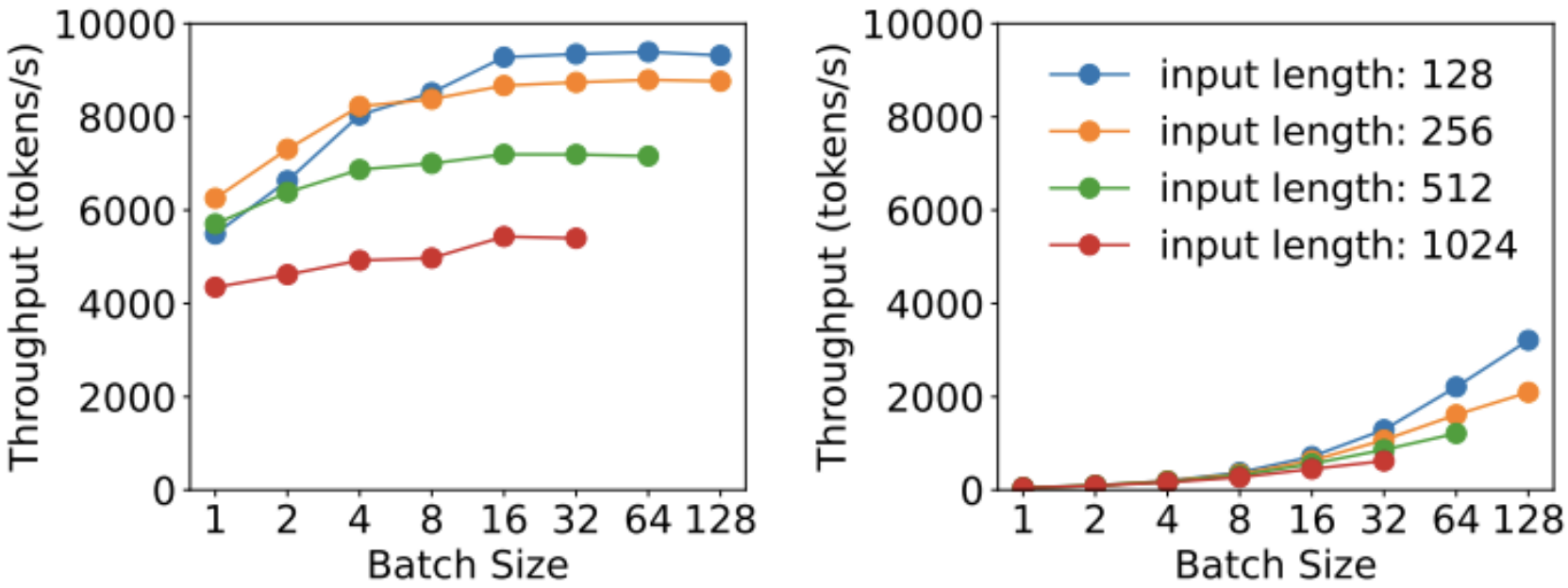


Figure 6: DistServe Runtime System Architecture

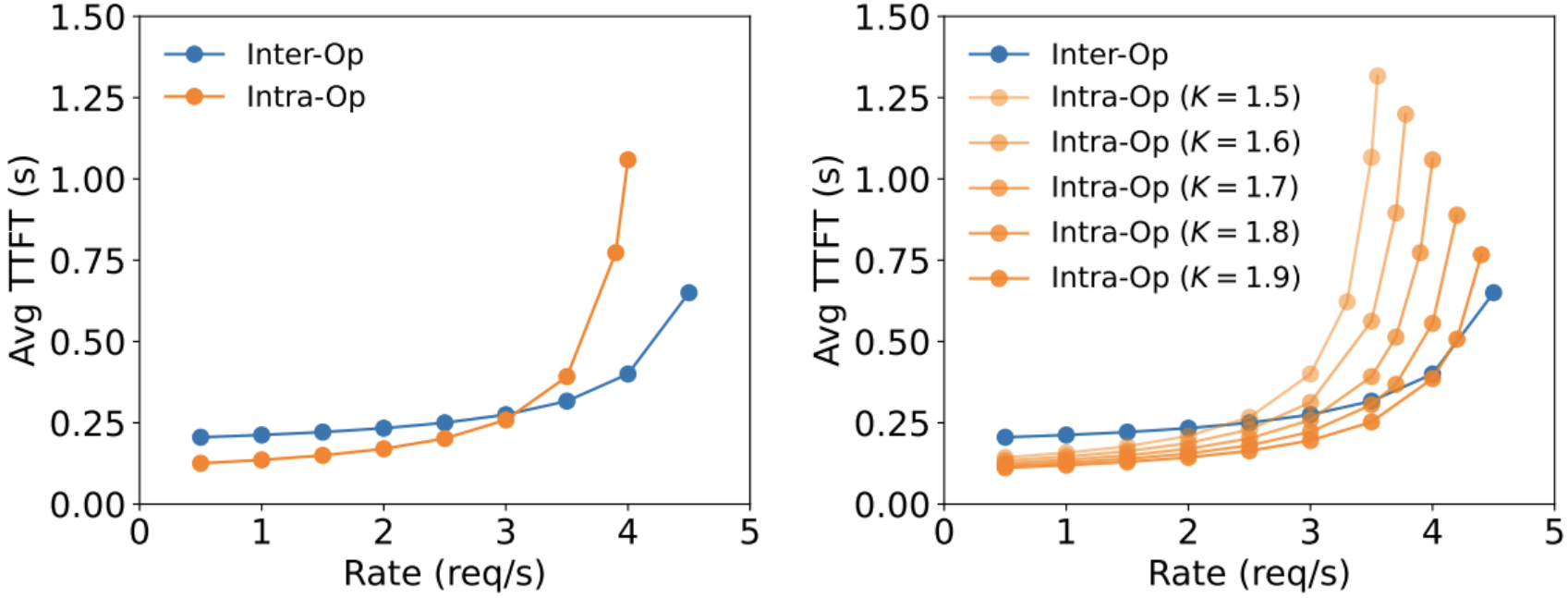
Profiling prefill and decode



(a) Prefill phase (b) Decoding phase

Figure 3: Throughput for two phases with different batch sizes and input lengths when serving an LLM with 13B parameters.

Profiling prefill



(a) Real experiment results (b) Changing intra-op speedup

Figure 4: Average TTFT when serving an LLM with 66B parameters using different parallelism on two A100 GPUs.

Profiling decode

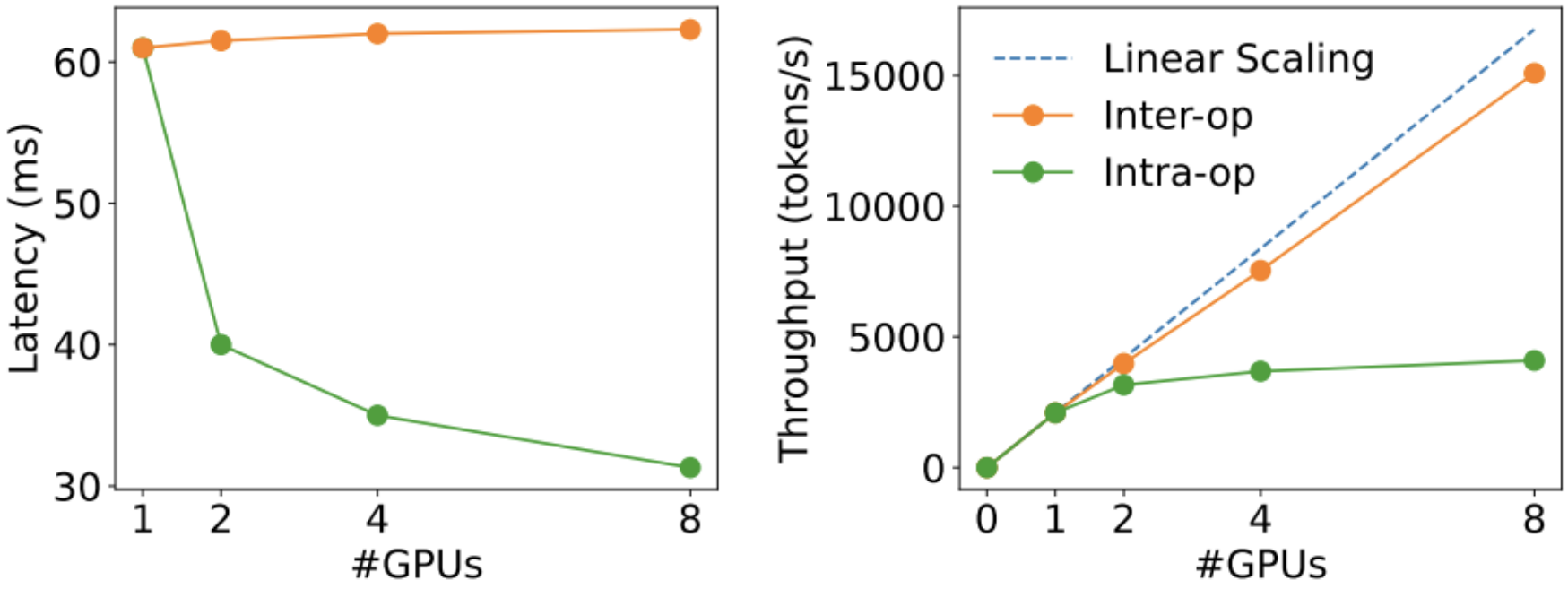


Figure 5: Decoding phase latency and throughput when serving a 13B LLM with batch size = 128 and input length = 256 under different parallel degrees.

Let’s check bandwidth needed

- The KV cache size of a single 512-token request on OPT-66B is approximately 1.13GB.

- Assuming an average arrival rate of 10 rps, we need to transfer 11.3GB data per second—or equivalently 90Gbps bandwidth to render the overhead invisible.

Placement for High Node-Affinity Cluster

Assumption:

- Nodes are connected with high bandwidth network, e.g., InfiniBand.

Observation:

- We can optimize prefill and decoding instances separately.

Algorithm Sketch:

- Use simulation to measure the goodput for a specific parallelism config.
- Obtain the optimal parallelism config for each phase.
- Use replication to match the overall traffic.

Algorithm 1 High Node-Affinity Placement Algorithm

Input: LLM G, #node limit per-instance N, #GPU per-node M, GPU memory capacity C, workload W, traffic rate R.

Output: the placement $best_plm$.

$config_p, config_d \leftarrow \emptyset, \emptyset$
for $intra_op \in \{1, 2, ..., M\}$ **do**
  for $inter_op \in \{1, 2, ..., \frac{N \times M}{intra_op}\}$ **do**
    if $\frac{G.size}{inter_op \times intra_op} < C$ **then**
      $config \leftarrow (inter_op, intra_op)$
      $\hat{G} \leftarrow \text{parallel}(G, config)$
      $config.goodput \leftarrow \text{simu_prefill}(\hat{G}, W)$
      if $\frac{config_p.goodput}{config_p.num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**
        $config_p \leftarrow config$
      $config.goodput \leftarrow \text{simu_decode}(\hat{G}, W)$
      if $\frac{config_d.goodput}{config_d.num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**
        $config_d \leftarrow config$
$n, m \leftarrow \lceil \frac{R}{config_p.goodput} \rceil, \lceil \frac{R}{config_d.goodput} \rceil$
$best_plm \leftarrow (n, config_p, m, config_d)$
return $best_plm$

Placement for Low Node-Affinity Cluster

Assumption:

- GPUs inside one node are connected with NVLINK.

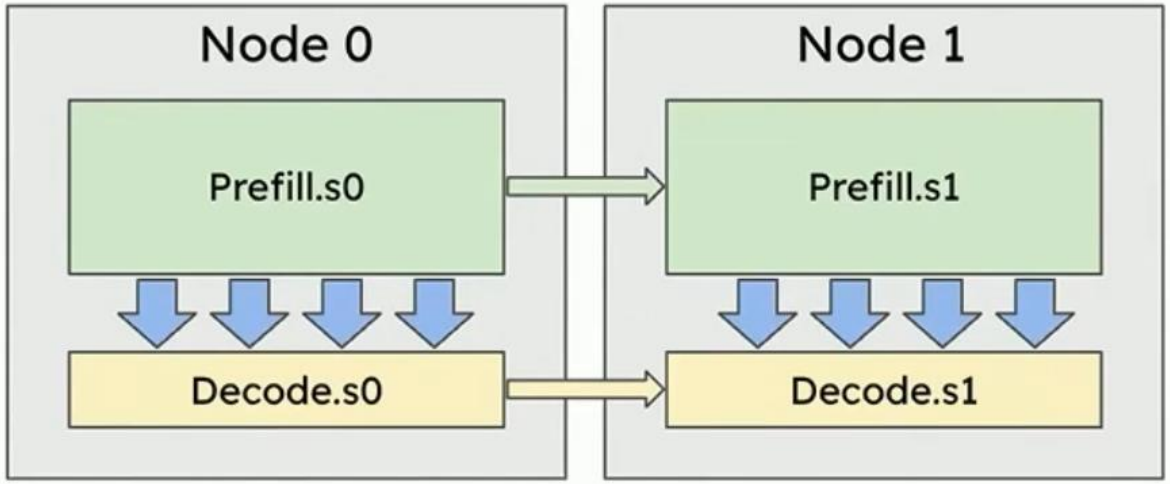
Observation:

- KV-Cache transmission only happens between ***the same layer.***

Algorithm Sketch:

- Similar to the previous one but add the constraint to require the same stage of prefill/decoding instances to be on the same node.

Example:



Algorithm 2 Low Node-Affinity Placement Algorithm

Input: LLM G, #node limit per-instance N, #GPU per-node M, GPU memory capacity C, workload W, traffic rate R.

Output: the placement *best_plm*.

$config^* \leftarrow \emptyset$
for $inter_op \in \{1, 2, ..., N\}$ **do**
  $\mathcal{P} \leftarrow$ get_intra_node_configs$(G, M, C, inter_op)$
  for $P_p \in \mathcal{P}$ **do**
    for $P_d \in \mathcal{P}$ **do**
      if $P_p.num_gpus + P_d.num_gpus \leq M$ **then**
        $config \leftarrow (inter_op, P_p, P_d)$
        $\hat{G}_p, \hat{G}_d \leftarrow \text{parallel}(G, config)$
        $config.goodput \leftarrow \text{simulate}(\hat{G}_p, \hat{G}_d, W)$
        if $\frac{config.^*goodput}{config.^*num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**
          $config^* \leftarrow config$
$n \leftarrow \lceil \frac{R}{config.^*goodput} \rceil$
$best_plm \leftarrow (n, config^*)$
return *best_plm*

Online scheduling

- Reducing pipeline bubbles
  - Make prefill length similar across batches
  - Make decode tokens the same per batch
- Combat burstiness
  - Pull-based, no push
- Replanning

SGLang: Efficient Execution of Structured Language Model Programs

Lianmin Zheng[2*] **Liangsheng Yin**[3] **Zhiqiang Xie**[1] **Chuyue Sun**[1] **Jeff Huang**[4]
Cody Hao Yu[5] **Shiyi Cao**[2] **Christos Kozyrakis**[1] **Ion Stoica**[2] **Joseph E. Gonzalez**[2]
Clark Barrett[1] **Ying Sheng**[1*]

[1] Stanford University [2] UC Berkeley [3] Shanghai Jiao Tong University
[4] Texas A&M University [5] Independent Researcher

Challenges of language model programs

- First, programming LM programs is tedious and difficult due to the non-deterministic nature of LLMs. Developing an LM program often requires extensive string manipulation, experimental tuning of prompts, brittle output parsing, handling multiple input modalities, and implementing parallelism mechanisms.

- Secondly and importantly, executing LM programs is inefficient due to redundant computation and memory usage.

```
dimensions = ["Clarity", "Originality", "Evidence"]

@function
def multi_dimensional_judge(s, path, essay):
  s += system("Evaluate an essay about an image.")
  s += user(image(path) + "Essay:" + essay)
  s += assistant("Sure!")

  # Return directly if it is not related
  s += user("Is the essay related to the image?")
  s += assistant(select("related", choices=["yes", "no"]))
  if s["related"] == "no": return

  # Judge multiple dimensions in parallel
  forks = s.fork(len(dimensions))
  for f, dim in zip(forks, dimensions):
    f += user("Evaluate based on the following dimension:" +
      dim + ". End your judgment with the word 'END'")
    f += assistant("Judgment:" + gen("judgment", stop="END"))

  # Merge the judgments
  judgment = "\n".join(f["judgment"] for f in forks)

  # Generate a summary and a grade. Return in the JSON format.
  s += user("Provide the judgment, summary, and a letter grade")
  s += assistant(judgment + "In summary," + gen("summary", stop=".")
                 + "The grade of it is" + gen("grade"))

  schema = r'\{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+-]?"\}'
  s += user("Return in the JSON format.")
  s += assistant(gen("output", regex=schema))

state = multi_dimensional_judge.run(...)
print(state["output"])
```

Handle chat template and multi-modal inputs

Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

Fetch generation results

Runtime optimization: API speculative execution (Sec. 5)

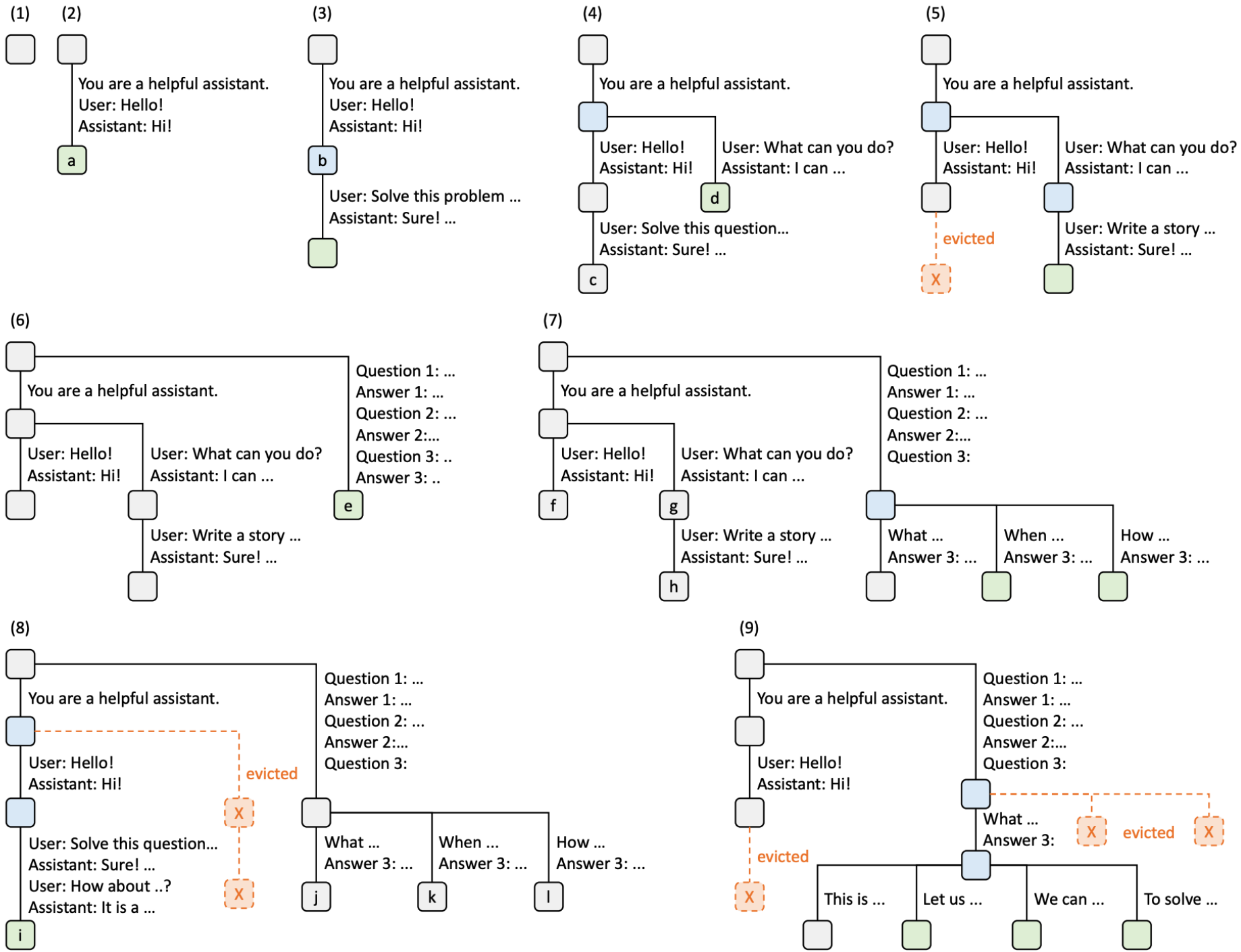
Runtime optimization: fast constrained decoding (Sec. 4)

Run an SGLang program

Figure 2: The implementation of a multi-dimensional essay judge in SGLang utilizes the branch-solve-merge prompting technique [40]. Primitives provided by SGLang are shown in red.

Efficient KV Cache Reuse with RadixAttention

Given the KV cache reuse opportunity, a key challenge in optimizing SGLang programs is reusing the KV cache across multiple calls and instances. While some systems explore certain KV cache reuse cases [23, 58, 18, 12], they often need manual configurations and cannot handle all reuse patterns (e.g., dynamic tree structures). Consequently, most state-of-the-art inference systems recompute the KV cache for each request. We will discuss their limitations and our differences in Sec. 7.


(1)
(2)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
a
(3)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
b
User: Solve this problem ...
Assistant: Sure! ...
(4)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
User: What can you do?
Assistant: I can ...
d
User: Solve this question...
Assistant: Sure! ...
c
(5)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
User: What can you do?
Assistant: I can ...
evicted
X
User: Write a story ...
Assistant: Sure! ...
(6)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
User: What can you do?
Assistant: I can ...
User: Write a story ...
Assistant: Sure! ...
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3: ..
Answer 3: ..
e
(7)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
f
User: What can you do?
Assistant: I can ...
g
User: Write a story ...
Assistant: Sure! ...
h
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3:
What ...
Answer 3: ...
When ...
Answer 3: ...
How ...
Answer 3: ...
(8)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
evicted
X
User: Solve this question...
Assistant: Sure! ...
User: How about ..?
Assistant: It is a ...
X
i
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3:
What ...
Answer 3: ...
j
When ...
Answer 3: ...
k
How ...
Answer 3: ...
l
(9)
You are a helpful assistant.
User: Hello!
Assistant: Hi!
evicted
X
Question 1: ...
Answer 1: ...
Question 2: ...
Answer 2:...
Question 3:
What ...
Answer 3:
X
evicted
X
This is ...
Let us ...
We can ...
To solve ...

Cache-aware scheduling

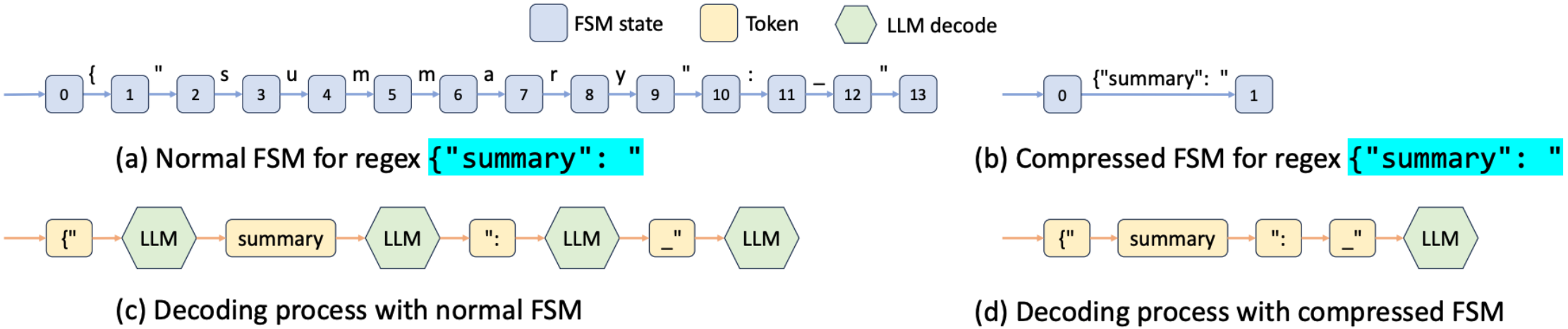
- Prioritize requests with longer matches prefixes instead of first-come, first-serve

Constrained decoding

- SGLang offers a regex argument to enforce such constraints using regular expressions, which are expressive enough for many practical scenarios.

- Existing systems support this by converting a regular expression into a finite state machine (FSM). During decoding, they maintain the current FSM state, retrieve allowed tokens from the next states, and set the probability of invalid tokens to zero, decoding token by token. This token-by-token approach, however, is inefficient when there are opportunities to decode multiple tokens at once.

FSM compression



(a) Normal FSM for regex `{"summary": "`

(b) Compressed FSM for regex `{"summary": "`

(c) Decoding process with normal FSM

(d) Decoding process with compressed FSM

Figure 4: The decoding process of normal and compressed FSMs (the underscore _ means a space).